ARCH TECHNOLOGIES

Pakistan's Digital AI Training Platform

Cyber Security Internship

Month 2 Task Report

Name	Hifz Asim Shayan Khan
Roll No	ARCH-2602-1232
Internship Program	Cyber Security
Task	Credit card Fraud Detection & Lost Data Retrieval

Project 1: Credit Card Fraud Detection

1.1 Project Overview

Domain	Financial Security / Machine Learning
Goal	Detect fraudulent credit card transactions using unsupervised anomaly detection
Dataset	creditcard.csv — real-world anonymised transaction data (PCA-transformed features)
Techniques	Isolation Forest, Local Outlier Factor (LOF), One-Class SVM
Language	Python 3 (Google Colab)

Credit card fraud represents a severe financial threat to both consumers and institutions. Because fraudulent transactions are rare relative to legitimate ones, the dataset is highly imbalanced. This project applies three unsupervised anomaly-detection algorithms that do not require labelled training data, making them well-suited for real-world fraud scenarios where labelled examples may be scarce.

1.2 Dataset & Class Distribution

The dataset contains transactions made by European cardholders. Each row represents one transaction with the following key columns:

- V1–V28: PCA-transformed numerical features (anonymised for privacy)
- Time: Seconds elapsed since the first transaction in the dataset
- Amount: Transaction value in euros
- Class: Ground-truth label — 0 = Normal, 1 = Fraud

Class imbalance is a defining characteristic of the data. The table below shows the approximate split:

Class	Label	Count (approx.)	Proportion
0	Normal	284,315	~99.83%
1	Fraud	492	~0.17%

Because fraudulent records make up only ~0.17% of all transactions, standard accuracy metrics can be misleading. A naive model that labels everything as normal achieves 99.83% accuracy while detecting zero fraud — hence the importance of precision, recall, and the confusion matrix.

```
# ========================
# Class Distribution Plot
# ========================
sns.countplot(x='Class', data=data)
plt.title("Transaction Class Distribution (0 = Normal, 1 = Fraud)")
plt.show()
```

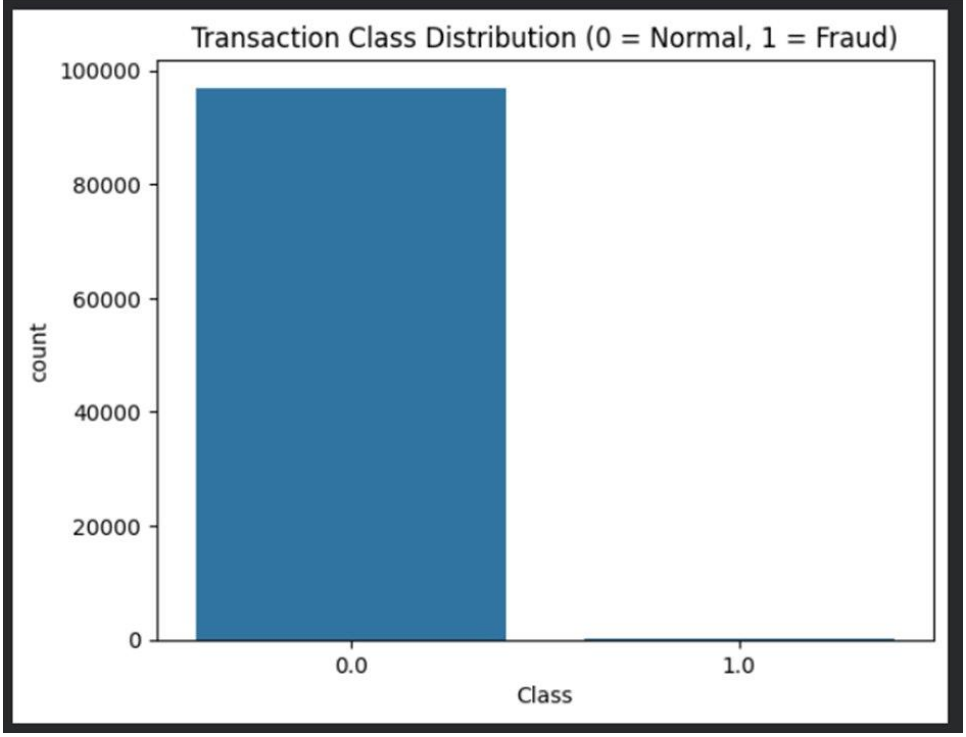

Figure 1: Transaction Class Distribution — the extreme imbalance between Normal (Class 0) and Fraud (Class 1) is clearly visible

1.3 Exploratory Data Analysis

Transaction Amount vs Time

A scatter plot of Time vs. Amount clearly shows that normal transactions span a wide value range while fraud points are concentrated at lower amounts and distributed across the full time window. This clustering at low amounts reflects real-world fraud patterns where criminals often test with small-value transactions.

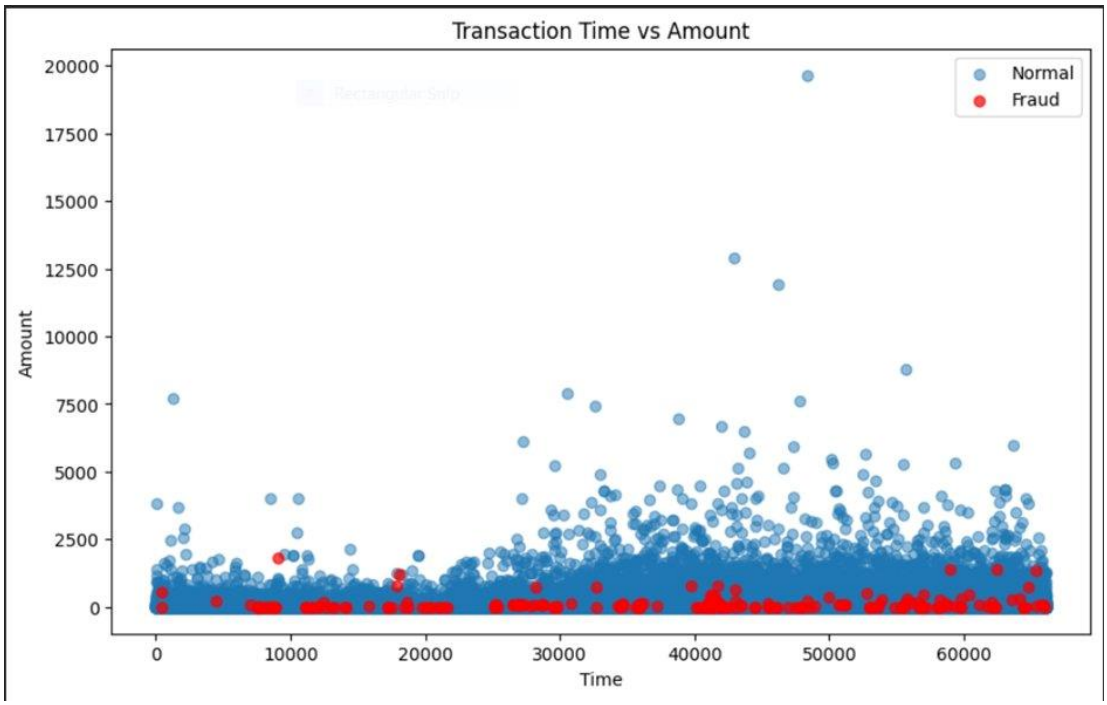

Figure 2: Transaction Time vs Amount — blue dots represent normal transactions; red dots represent fraudulent ones

Correlation Analysis

A 10% stratified sample is drawn for correlation analysis and modelling to keep computation tractable. The heatmap below visualises pairwise relationships between all 30 features (V1–V28, Time, Amount). Most PCA features are intentionally uncorrelated by design. The strong diagonal confirms each feature correlates perfectly with itself. Notably, several V-features and the Class column show meaningful off-diagonal colour, revealing subtle predictive signal.

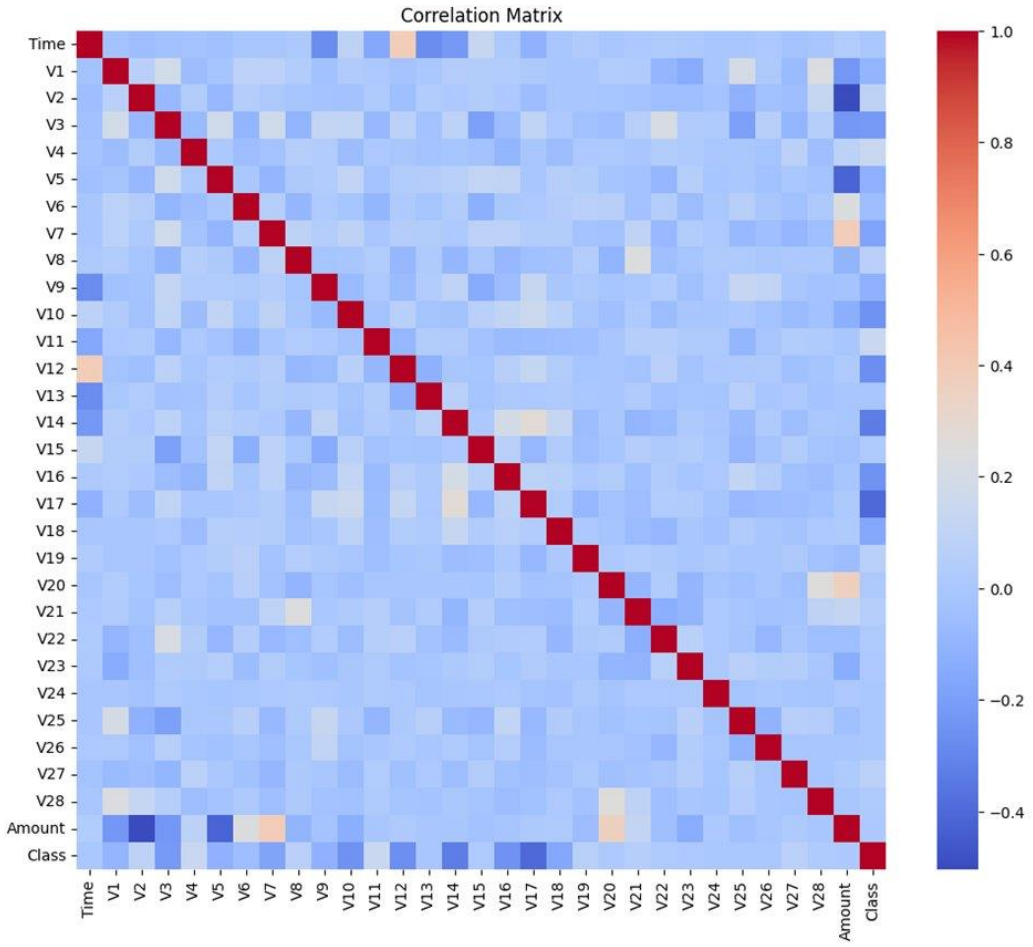

Figure 3: Feature Correlation Matrix — red indicates positive correlation, blue indicates negative correlation

1.4 Methodology

Three algorithms are evaluated. Each is an unsupervised method — trained without using the Class labels — that identifies observations deviating significantly from the majority pattern:

Algorithm	Core Idea	Key Parameters
Isolation Forest	Randomly partitions feature space; anomalies isolated in fewer splits	n_estimators=100, contamination=outlier_fraction
Local Outlier Factor (LOF)	Compares local density to neighbours; low-density points are outliers	n_neighbors=20, contamination=outlier_fraction
One-Class SVM	Learns a tight decision boundary around normal data in kernel space	kernel=rbf, gamma=0.1, nu=0.05

The contamination parameter is set to the actual fraud rate in the sample (len(Fraud) / len(Valid)), providing each model with a realistic expectation of the anomaly proportion.

1.5 Implementation Highlights

Sampling

```
data1 = data.sample(frac=0.1, random_state=42)
Fraud  = data1[data1['Class'] == 1]
Valid  = data1[data1['Class'] == 0]
outlier_fraction = len(Fraud) / float(len(Valid))
```

Label Conversion

scikit-learn anomaly detectors output +1 (inlier) and -1 (outlier). These are remapped to match the dataset convention where 1 = fraud:

```
y_pred[y_pred ==  1] = 0   # inlier  -> normal
y_pred[y_pred == -1] = 1   # outlier -> fraud
```

LOF Special Case

Unlike Isolation Forest and One-Class SVM, LOF does not expose a separate predict() method — predictions are obtained directly from fit_predict():

```
if name == 'Local Outlier Factor':
    y_pred = clf.fit_predict(X)
else:
    clf.fit(X)
    y_pred = clf.predict(X)
```

1.6 Results & Evaluation

Each model is evaluated using overall accuracy plus a full classification report (precision, recall, F1-score per class). Confusion matrix heatmaps below provide a detailed visual breakdown of true positives, false positives, true negatives, and false negatives for each model.

Isolation Forest

Isolation Forest achieved the highest overall accuracy (~99.78%) with a fraud recall of 0.38, detecting 6 out of 16 fraud cases. It produced only 11 false positives from 9,698 normal transactions — an excellent precision on the normal class.

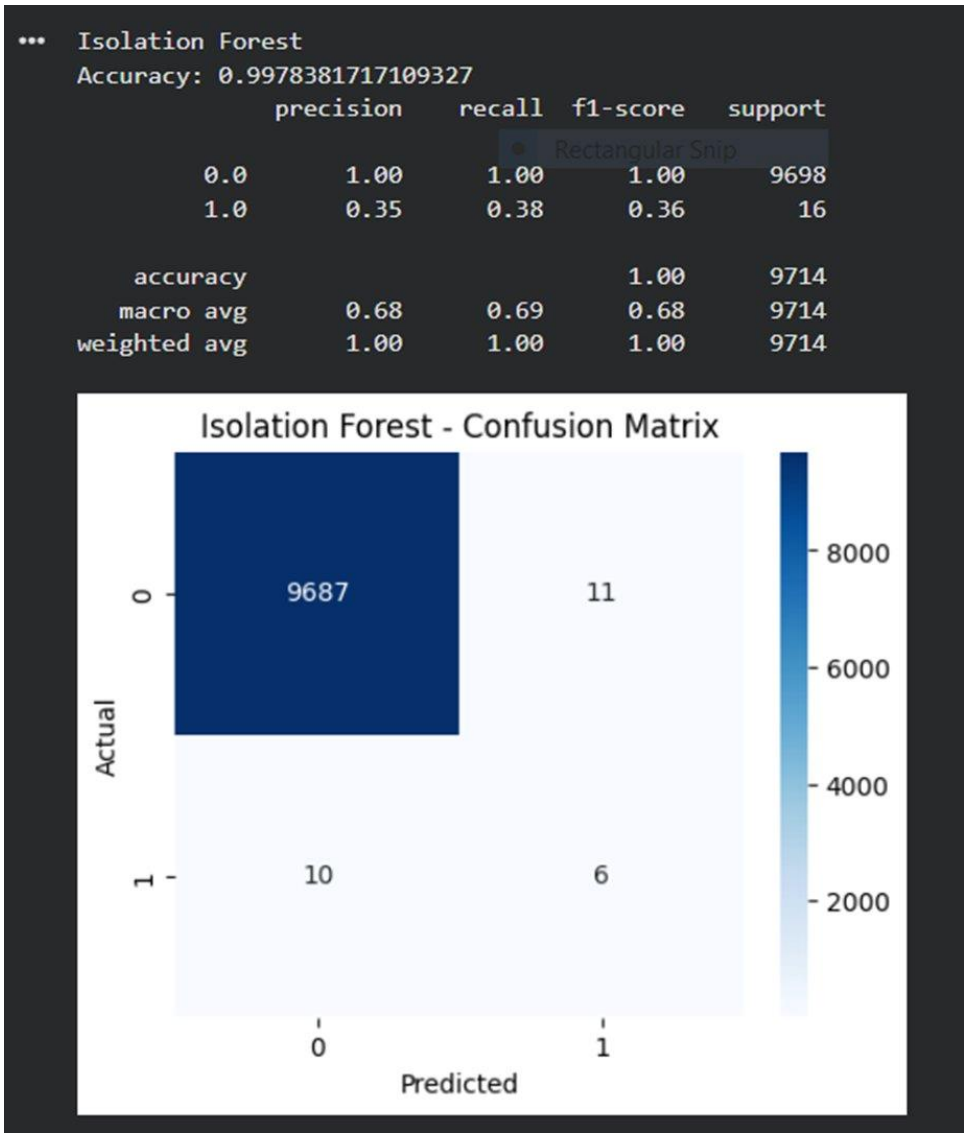

Figure 4: Isolation Forest — Classification Report & Confusion Matrix

Local Outlier Factor (LOF)

LOF achieved 99.66% accuracy but failed to detect any fraud cases — recall of 0.00 on Class 1. While the high overall accuracy looks impressive, this model is effectively useless for fraud detection as it labelled all transactions as normal.

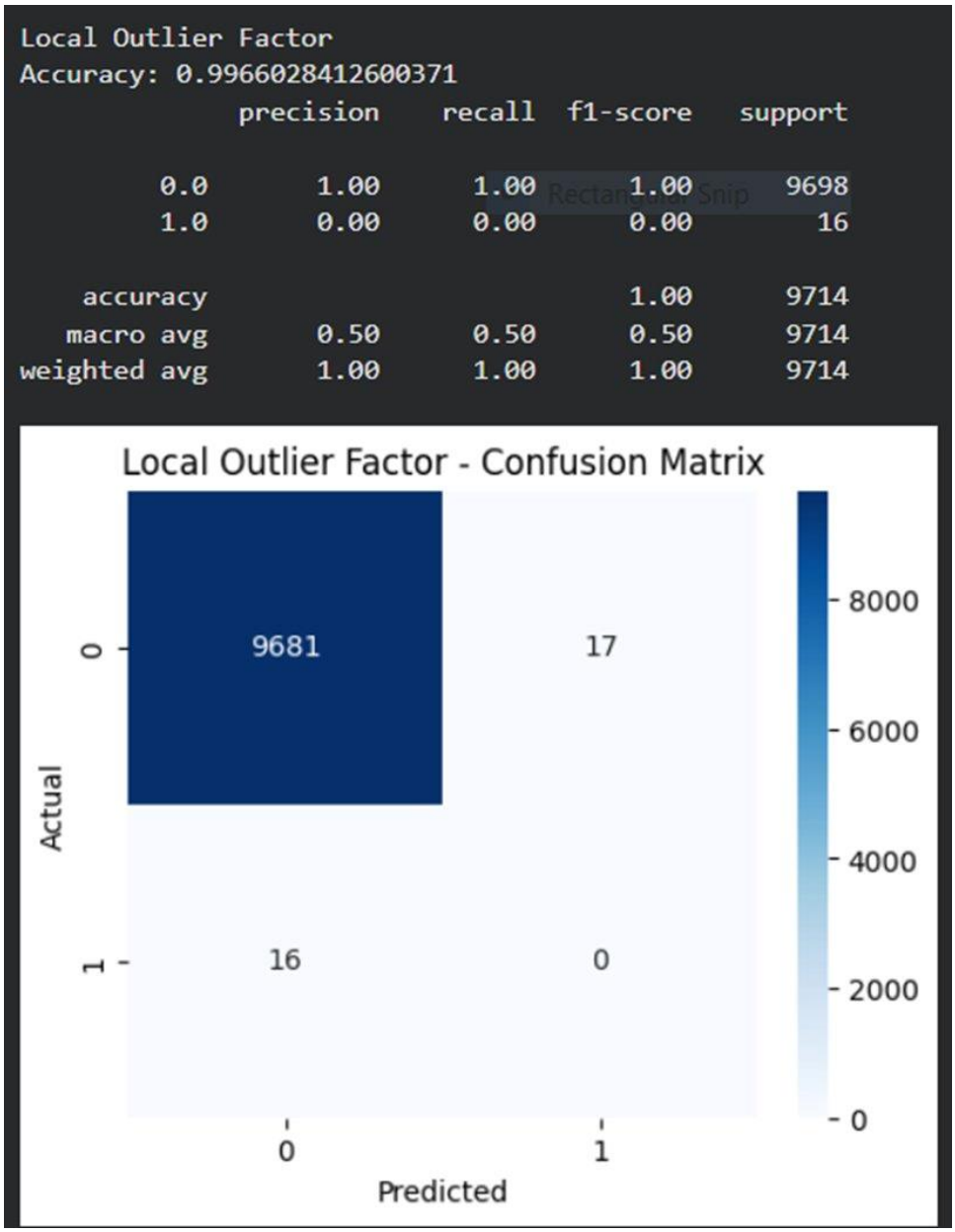

Figure 5: Local Outlier Factor — Classification Report & Confusion Matrix

One-Class SVM

One-Class SVM achieved only 66.6% overall accuracy due to 3,230 false positives — it flagged roughly one third of all normal transactions as fraud. However it correctly identified 3 out of 16 fraud cases. The poor performance reflects sensitivity to the RBF kernel hyperparameters (gamma, nu).

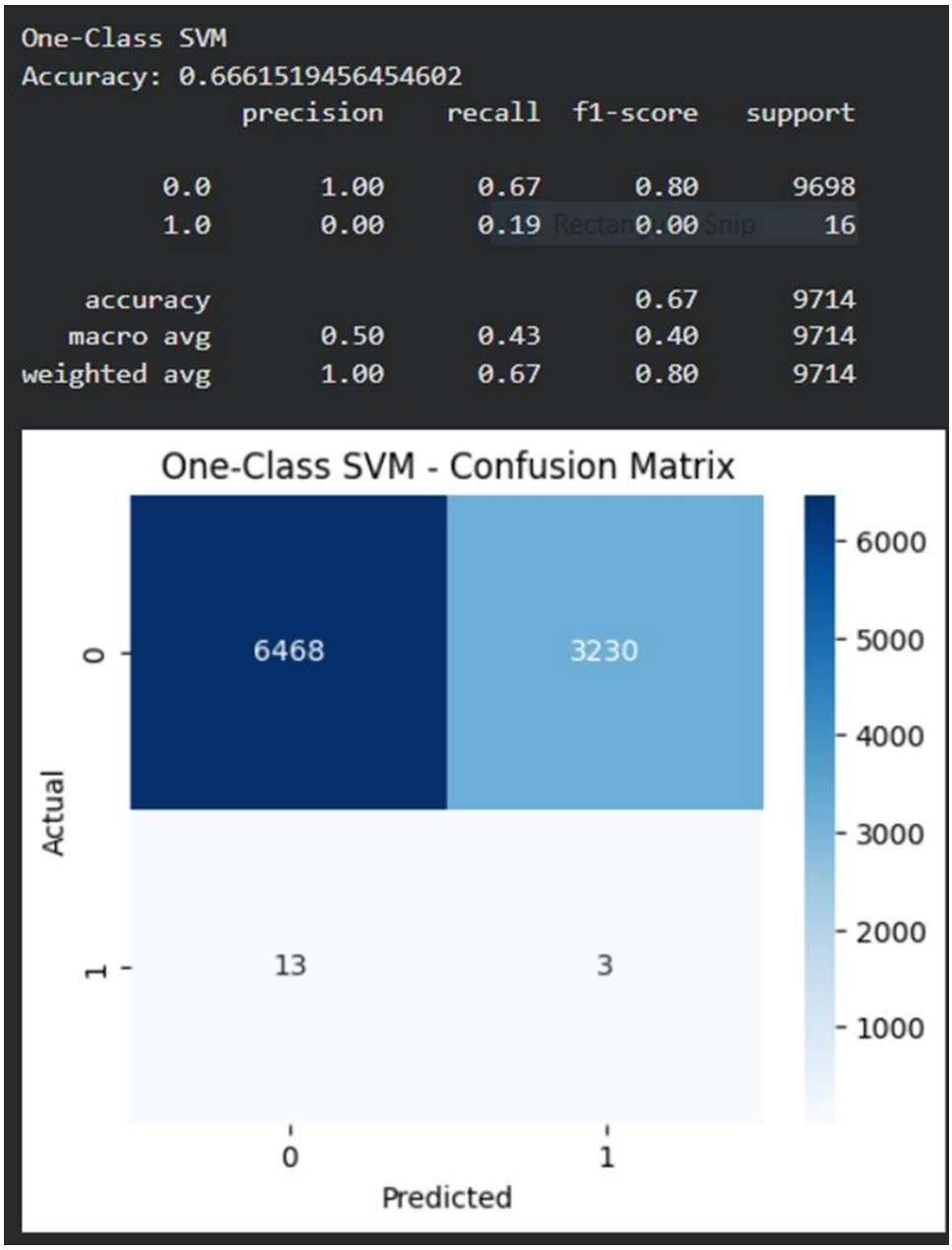

Figure 6: One-Class SVM — Classification Report & Confusion Matrix

Model Accuracy Comparison

The bar chart below summarises overall accuracy across all three models. While accuracy alone favours Isolation Forest and LOF equally, the confusion matrices reveal that LOF completely fails on fraud recall — making Isolation Forest the clear winner for this task.

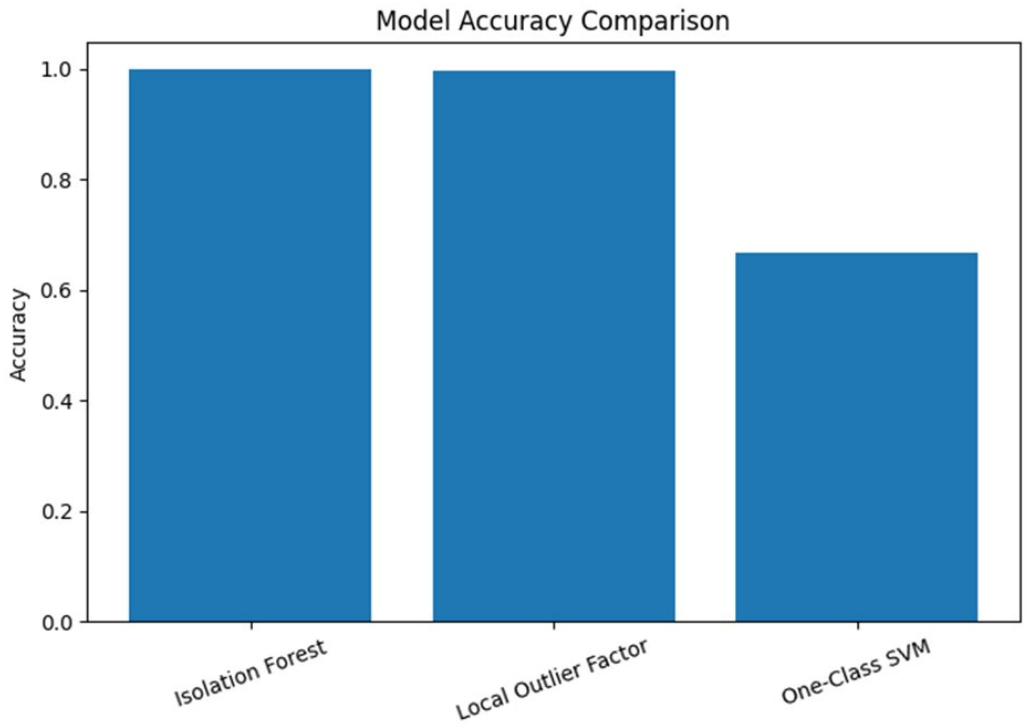

Figure 7: Model Accuracy Comparison — note that high accuracy does not imply effective fraud detection

Model	Accuracy	Fraud Recall	False Positives	Verdict
Isolation Forest	99.78%	0.38 (6/16)	11	Best overall
Local Outlier Factor	99.66%	0.00 (0/16)	17	Fails on fraud
One-Class SVM	66.61%	0.19 (3/16)	3,230	Unusable

1.7 Key Takeaways

- Accuracy alone is insufficient for imbalanced classification — recall on the fraud class is the critical metric.
- LOF scored 99.66% accuracy but detected zero fraud; Isolation Forest is the superior choice despite similar headline accuracy.
- The contamination parameter significantly affects model sensitivity; it should be tuned based on domain knowledge.
- Unsupervised methods are practical when labelled data is limited or unavailable.
- Future work could include supervised models (XGBoost, Random Forest) with SMOTE oversampling for comparison.

Project 2: JPEG Data Recovery Tool

2.1 Project Overview

Domain	Digital Forensics / Systems Programming
Goal	Recover deleted or lost JPEG images from raw disk/storage device data
Technique	File carving — signature-based JPEG header/footer detection
Language	Python 3
Input	Raw block device or disk image (e.g. /dev/sdb, .img file)

When files are deleted from a storage device, the filesystem metadata (directory entries, allocation tables) is removed, but the underlying binary data often remains intact on the physical medium. File carving is the process of scanning raw binary data sector by sector to identify and extract file content based on known binary signatures — without relying on any filesystem structure.

This tool implements JPEG file carving in pure Python, making it portable and dependency-free. It is applicable to USB drives, SD cards, hard disk images, and forensic disk dumps.

2.2 JPEG File Format Background

JPEG files follow a well-defined binary structure that makes them ideal candidates for file carving:

Marker	Hex Bytes	Meaning
SOI (Start of Image)	FF D8 FF E0 00 10 4A 46	Marks the beginning of a JPEG file
EOI (End of Image)	FF D9	Marks the end of a JPEG file
JFIF Header	4A 46 49 46 (JFIF)	Standard JFIF application marker following SOI

The tool searches for the 8-byte SOI + JFIF marker sequence to locate JPEG starts, and the 2-byte EOI marker to locate JPEG ends. All bytes between a matched SOI and its corresponding EOI are written to a recovered file.

2.3 Algorithm & Implementation

High-Level Algorithm

- Open the target device/image in raw binary read mode.

- Read the device sector by sector (default sector size: 512 bytes).
- Scan each sector for the JPEG SOI header signature.
- When a header is found, open a new output file and begin writing bytes.
- Continue reading sectors and writing to the output file.
- When the JPEG EOI footer is detected, finalise and close the output file.
- Increment the recovered file counter and resume scanning.

Core Implementation

```
drive = "/dev/sdb"          # Target device or disk image
fileD = open(drive, "rb")

size  = 512                 # Sector size in bytes
byte  = fileD.read(size)
offs  = 0                   # Sector offset counter
drec  = False               # Currently recovering a JPEG?
rcvd  = 0                   # Number of files recovered

JPEG_HEADER = b'\xff\xd8\xff\xe0\x00\x10\x4a\x46'
JPEG_FOOTER = b'\xff\xd9'
```

Detection & Extraction Loop

```
while byte:
    found = byte.find(JPEG_HEADER)
    if found >= 0:
        drec  = True
        print('Found JPG at:', hex(found + (size * offs)))
        fileN = open(str(rcvd) + '.jpg', 'wb')
        fileN.write(byte[found:])

        while drec:              # Read until EOI found
            byte  = fileD.read(size)
            bfind = byte.find(b'\xff\xd9')
            if bfind >= 0:
                fileN.write(byte[:bfind + 2])
                fileN.close()
                rcvd += 1
                drec  = False
            else:
                fileN.write(byte)

    byte = fileD.read(size)
    offs += 1

fileD.close()
```

2.4 Key Design Decisions

Decision	Rationale
512-byte sector reads	Matches standard disk sector size; minimises memory usage while reading large devices

Byte-level find()	Python's built-in bytes.find() is optimised for pattern matching in binary buffers
Offset tracking (offs)	Logs the exact disk address of each recovered file for forensic chain-of-custody documentation
Sequential file naming	Simple, collision-free naming for batch recovery (0.jpg, 1.jpg...)
Raw binary mode (rb)	Ensures no newline translation or encoding interference with binary data

2.5 Limitations & Potential Improvements

Current Limitations

- Fragment handling: If a JPEG is stored in non-contiguous sectors (fragmented), recovery may be incomplete or corrupt.
- Signature scope: Only standard JFIF JPEGs are targeted; EXIF-header JPEGs (FF D8 FF E1) are not detected.
- No error handling: If the device is disconnected mid-read, the script will raise an unhandled exception.
- Single footer match: The EOI marker FF D9 can appear within JPEG data (e.g., as a Huffman code), potentially causing premature file closure.

Suggested Enhancements

- EXIF support: Add detection for FF D8 FF E1 (EXIF header) alongside JFIF.
- Error handling: Wrap file I/O in try/except blocks and implement graceful shutdown on read errors.
- Output directory: Write recovered files to a configurable output folder rather than the working directory.
- Progress reporting: Display a progress bar (e.g., using tqdm) for large devices.
- File validation: Verify recovered JPEGs using Pillow (PIL) to filter out corrupt carves.

2.6 Usage Instructions

Prerequisites

```
# No external dependencies required — pure Python 3
# Run with appropriate permissions to access raw devices:
sudo python3 Data_Recovery.py
```

Configuration

```
drive = "/dev/sdb"    # Change to your target device or image path
size  = 512           # Adjust for 4K sector drives if needed (use 4096)
```

Expected Output

```
==== Found JPG at location: 0x3a200 ====
==== Wrote JPG to file: 0.jpg ====

==== Found JPG at location: 0x6f400 ====
==== Wrote JPG to file: 1.jpg ====
```

2.7 Key Takeaways

- File carving is a foundational digital forensics technique that bypasses filesystem-level damage entirely.
- Known file signatures (magic bytes) enable reliable detection without metadata.
- Sector-level I/O in Python provides direct, efficient access to raw storage media.
- This tool demonstrates how even a short script can have significant real-world forensic utility.
- Extensions toward a full forensic toolkit could include support for PNG, PDF, ZIP, MP4, and other recoverable formats.

End of Portfolio Document